

USE REF() HOOK - COMPLETE NOTES

Page: _____

1. What is useRef() ?

- `useRef()` is a React Hook that returns a mutable ref object.
- It allows you to persist values between renders without causing re-renders.
- Useful for accessing DOM elements, storing mutable values, timers, previous values, etc.

Mutable value
container
that survives
re-renders



2. Why use useRef() ?

- ✓ Access DOM elements directly.
- ✓ Store mutable values (does not cause re-render).
- ✓ Keep track of previous values.
- ✓ Manage timers, intervals, timeouts.
- ✓ Avoid unnecessary re-renders.

`useRef()`
= Persistent
+ Mutable
+ No Re-render



3. Basic Syntax

```
import { useRef } from 'react';

const myRef = useRef(initialValue);

// myRef is an object
// myRef.current holds the value
```



```
myRef
{
  current: initialValue
}
```

4. Important Points

- `useRef()` returns the same object on every render.
- Changing `ref.current` does NOT trigger a re-render.
- The `.current` property is mutable.
- The initial value is used only on the first render.

Same ref
object across
all renders



5. Accessing DOM Elements

```
import { useRef } from 'react';

function FocusInput() {
  const inputRef = useRef(null);

  const handleClick = () => {
    inputRef.current.focus();
  };

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Type..." />
      <button onClick={handleClick}>Focus Input</button>
    </div>
  );
}
```

Most common
use of `useRef`
→ Access DOM
directly



How it works ?

`useRef(null)`



ref object
{ current: null }



attached to
DOM element



`ref.current`
points to DOM

6. Storing Mutable Values (No Re-render)

```
import { useRef } from 'react';
function ClickCounter() {
  const countRef = useRef(0); // does not cause re-render

  const handleClick = () => {
    countRef.current = countRef.current + 1;
    console.log("Count:", countRef.current);
  };

  return <button onClick={handleClick}>Click Me </button>;
}
```

Changing
ref.current
does not
re-render
component



7. Previous Value Example

```
import { useRef, useState, useEffect } from 'react';
function ShowPreviousValue() {
  const [count, setCount] = useState(0);
  const prevCountRef = useRef();

  useEffect(() => {
    prevCountRef.current = count; // store previous value
  }, [count]);

  return (
    <div>
      <p>Current Count: {count}</p>
      <p>Previous Count: {prevCountRef.current}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
```

useRef helps
store previous
value without
extra re-render



8. useRef with setTimeout / setInterval

```
function Timer() {
  const timerRef = useRef(null);

  const startTimer = () => {
    timerRef.current = setTimeout(() => {
      alert("Time is up!");
    }, 3000);
  };

  const clearTimer = () => {
    clearTimeout(timerRef.current);
  };

  return (
    <div>
      <button onClick={startTimer}>Start </button>
      <button onClick={clearTimer}>Cancel </button>
    </div>
  );
}
```

Store timer/id
in ref so we
can clear it
later



9. Common useRef() Use Cases

- Focus management
- DOM measurements
- Store timers / intervals
- Previous values
- Persist values without re-render
- Scroll position
- Video / Audio control

Many real-life
scenarios in
React apps



10. Difference: useRef vs useState

<u>Feature</u>	<u>useState</u>	<u>useRef</u>
Purpose	Manage state	Stores mutable values / DOM reference
Triggers Re-render	Yes	No
Update	Using <code>setState()</code>	Directly change <code>ref.current</code>
Syntax	<code>const [state, setState] = useState(initial)</code>	<code>const ref = useRef(initial)</code>
Use Cases	UI data that affects render	DOM access, timers, previous values, etc.

11. Important Notes

- ✓ Do not use `useRef` for data that affects UI.
- ✓ Do not read or write `ref.current` during rendering (except initial use).
- ✓ Use `ref` with `forwardRef` to pass refs to child components.
- ✓ `useRef` is perfect for performance optimizations.

Use smartly
for better
performance

12. Quick Revision

- `useRef()` returns a `ref` object with `.current` property.
- `.current` is mutable and does not cause re-render.
- Use `ref` for DOM access, timers, previous values.
- Use `useState` for UI-related state.
- Same `ref` object persists across renders.

Complete Flow13. Mini Example Summary

```

const inputRef = useRef(null);
<input ref={inputRef} />

inputRef.current.focus(); // access DOM
inputRef.current = 10; // store value
inputRef.current = "New"; // update without re-render
  
```

One Hook
Many
Possibilities



★ `useRef()` = Mutable + Persistent + No Re-render = Power Tool ★

CUSTOM HOOKS - COMPLETE NOTES

Page

1. What is a Custom Hook?

- A Custom Hook is a JavaScript function whose name starts with "use" and that may call other Hooks.
- It lets you extract and reuse component logic.
- It does not work inside classes (only in functional components).

Reusability
Clean Code
Separation of
Concerns



2. Why Custom Hooks?

- ✓ Reuse logic across components.
- ✓ Keep components clean and focused on UI.
- ✓ Easier testing and maintenance.
- ✓ Share stateful logic without Redux/Zustand (when not needed).

3. Basic Syntax

```
function useSomething ( params ) {  
  // use state, effect, context, etc.  
  return {  
    // return values / functions  
  };  
}
```

Must start
with "use"
→
useSomething

4. Rules of Custom Hooks

- Only call Hooks at the top level.
- Do not call Hooks inside loops, conditions, or nested functions.
- Only call Hooks from React function components or other Custom Hooks.

Follow the
Rules of Hooks
Always



5. Simple Example - useCounter

```
import { useState } from 'react'  
function useCounter(initial = 0) {  
  const [count, setCount] = useState(initial);  
  
  const increment = () => setCount(c => c + 1);  
  const decrement = () => setCount(c => c - 1);  
  const reset = () => setCount(initial);  
  
  return { count, increment, decrement, reset };  
}
```

We return
values and
functions
to use in
components.

6. Using the Custom Hook

```
function Counter() {  
  const { count, increment, decrement, reset } = useCounter(10);  
  
  return (  
    <div>  
      <h2>Count: {count}</h2>  
      <button onClick={increment}>+1</button>  
      <button onClick={decrement}>-1</button>  
      <button onClick={reset}>Reset</button>  
    </div>  
  );  
}
```

Component
gets clean &
focused only
on UI.

★ Think of Custom Hooks as a tool box for your logic.
Use it, reuse it, and keep your components simple!



7. Using useEffect Inside Custom Hook

```
import { useState, useEffect } from 'react'
function useWindowSize() {
  const [size, setSize] = useState({ width: window.innerWidth,
    height: window.innerHeight });

  useEffect(() => {
    function handleResize() {
      setSize({ width: window.innerWidth, height: window.innerHeight });
    }
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return size; // { width, height }
}
```

Custom Hook
can handle
side effects
too



8. Using useRef Inside Custom Hook - usePrevious

```
import { useRef, useEffect } from 'react'
function usePrevious(value) {
  const ref = useRef();
  useEffect(() => {
    ref.current = value;
  }, [value]);
  return ref.current;
}
```

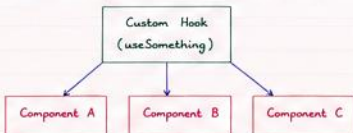
Returns
previous value
of something
between
renders.

9. Custom Hook with Input - useToggle

```
import { useState, useCallback } from 'react'
function useToggle(initial = false) {
  const [value, setValue] = useState(initial);
  const toggle = useCallback(() => setValue(v => !v), []);
  return [value, toggle]; // Array return style
}
```

Returns array
(like built-in
hooks).

10. Sharing Logic Between Multiple Components



One hook,
many
components.



11. When to use Custom Hooks ?

- When same logic is used in multiple components.
- When component becomes too big and mixed with too much logic.
- When you want to separate logic (state, effect, calculations) from UI.
- When you want your code to be more maintainable and testable.

Mini Summary

- Custom Hook → Reusable logic
- Start with "use" → Rule
- Can use other Hooks → useState, useEffect, useRef, etc.
- Return values / functions → Use in components
- Keep components clean → Better structure ✨

12. Advanced Example - useFetch

```
import { useState, useEffect } from 'react'

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  useEffect(() => {
    let isMounted = true;
    async function fetchData() {
      try {
        setLoading(true);
        const res = await fetch(url);
        const json = await res.json();
        if (isMounted) setData(json);
      } catch (err) {
        if (isMounted) setError(err.message);
      } finally {
        if (isMounted) setLoading(false);
      }
    }
    fetchData();
    return () => { isMounted = false }
  }, [url]);

  return { data, loading, error };
}
```

Reusable
data fetching
logic with
loading &
error states.

13. Custom Hook Best Practices

- ✓ Keep it focused - one hook, one job.
- ✓ Use meaningful names.
- ✓ Accept parameters if needed.
- ✓ Return only what is needed.
- ✓ Handle cleanup in useEffect.
- ✓ Document your hook.

Good Custom
Hooks =
Happy Future
You

14. Quick Revision

1. A Custom Hook is a function starting with "use".
2. Use other Hooks inside it.
3. Helps in reusability and clean code.
4. Can return values, functions or arrays.
5. Follow Rules of Hooks strictly.
6. Use when logic is repeated or component is messy.
7. Example Hooks → useCounter, useToggle, useFetch, usePrevious.

Complete Flow

REACT ROUTER (v6) - COMPLETE NOTES Page

1. What is React Router ?

- React Router is the standard routing library for React.
- It helps us to build Single Page Applications (SPA) with multiple views without page reload.
- It keeps the UI in sync with the URL.

Navigate
Without
Reloading
the page



2. Why React Router ?

- ✓ Better User Experience (no page reload)
- ✓ Bookmarkable URLs
- ✓ Protect Routes (Private / Public)
- ✓ Dynamic Routing
- ✓ Code Splitting & Lazy Loading
- ✓ Nested Routes

3. Installation

```
npm install react-router-dom
```

We use
react-router-dom
(not just
react-router)
in v6

4. Basic Setup

```
// main.jsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App'

ReactDOM.createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
)
```

BrowserRouter
wraps the
entire app
(only once)

5. Routes & Route (Basic)

```
// App.jsx
import { Routes, Route } from 'react-router-dom'
import Home from './pages/Home'
import About from './pages/About'
import Contact from './pages/Contact'

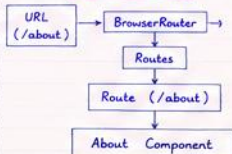
function App() {
  return (
    <Routes>
      <Route path="/" element={ <Home /> } />
      <Route path="/about" element={ <About /> } />
      <Route path="/contact" element={ <Contact /> } />
    </Routes>
  )
}
```

In v6
element={ }
is used
instead of
component=

Note:

- Only one <Route> is rendered at a time (best match).
- <Routes> must wrap all <Route>.

How it Works ?



6. Navigation (Link & NavLink)

<u>Link (Simple)</u>	<u>NavLink (With Active Style)</u>
<pre>import { Link } from 'react-router-dom' <Link to="/">Home</Link> <Link to="/about">About</Link> → Used for simple navigation.</pre>	<pre>import { NavLink } from 'react-router-dom' <NavLink to="/about" className={({ isActive }) => isActive ? 'active-link' : 'link' > > About </NavLink> → Useful to show active link.</pre>

CSS Example:

```
.link {
  color: blue;
  text-decoration: none;
  margin: 0 10px;
}

.active-link {
  color: red;
  font-weight: bold;
  border-bottom: 2px solid red;
}
```

7. useNavigate() Hook (Programmatic Navigation)

```
import { useNavigate } from 'react-router-dom'
function Login() {
  const navigate = useNavigate();
  const handleLogin = () => {
    navigate('/dashboard'); // redirect after login
  }
  return ( <button onClick={handleLogin}>Login </button>
)
```

useNavigate()
replaces
useHistory()
in v6

8. useParams() Hook (Dynamic Routes)

```
import { useParams } from 'react-router-dom'
function User() {
  const { id } = useParams(); // get :id from URL
  return ( <h2>User ID: {id} </h2>
)
```

Route Example:

```
<Route
  path="/user/:id"
  element=<User />
 />
```

9. useLocation() Hook

```
import { useLocation } from 'react-router-dom'
function ShowLocation() {
  const location = useLocation();
  return (
    <div>
      <p>Pathname: {location.pathname}</p>
      <p>Search: {location.search}</p>
      <p>Hash: {location.hash}</p>
    </div>
  )
}
```

Useful for
getting current
URL info
(pathname,
search, hash)

Common Hooks Summary

useNavigate()	→ For navigation
useParams()	→ For dynamic route params
useLocation()	→ For current URL info
useMatch()	→ To match a path
useSearchParams()	→ To read/set query params

Example URL

/user/42?name=kapil#top

pathname → /user/42

search → ?name=kapil

hash → #top

10. Nested Routes

```

<Route path="/" element={<Layout />}>
  <Route index element={<Home />} />  { * default child */}
  <Route path="/about" element={<About />} />
  <Route path="/contact" element={<Contact />} />
</Route>

```

```

// In Layout.jsx
import { Outlet } from 'react-router-dom'
function Layout() {
  return (
    <div>
      <Navbar />
      <Outlet />    { * child routes will render here */}
      <Footer />
    </div>
  )
}

```

<Outlet />
is used to
render child
routes

11. Protected / Private Route (Simple Way)

```

function PrivateRoute({ children }) {
  const isLoggedInIn = localStorage.getItem('token');
  if (!isLoggedInIn) {
    return <Navigate to="/login" replace />
  }
  return children;
}

<Route path="/dashboard" element={
  <PrivateRoute>
    <Dashboard />
  </PrivateRoute>
} />

```

Navigate

- Used to redirect
- replace → prevents going back to previous page

12. Lazy Loading (Code Splitting)

```

const Home = lazy(() => import('../pages/Home'))
const About = lazy(() => import('../pages/About'))

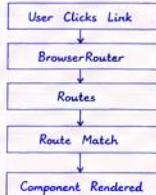
<Suspense fallback={<h2>Loading...</h2>}>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
  </Routes>
</Suspense>

```

Improves
performance
by loading
components
on demand

Quick Revision

- ✓ React Router helps in navigation without page reload.
- ✓ Use <BrowserRouter> at the root.
- ✓ Define routes using <Routes> & <Route>.
- ✓ Use Link / NavLink for navigation.
- ✓ useNavigate() for programmatic navigation.
- ✓ useParams() for dynamic params.
- ✓ useLocation() for current URL info.
- ✓ Nested routes use <Outlet />.
- ✓ Protect routes using <PrivateRoute> + <Navigate>.
- ✓ Lazy loading improves performance.

Complete Flow

REACT HOOK FORM + ZOD - COMPLETE NOTES

Page

1. What is React Hook Form ?

- React Hook Form (RHF) is a performant, flexible and lightweight library for building forms in React.
- Uses Uncontrolled Components + Refs internally for better performance.

Less Re-render
Better Performance
Easy Validation
DX



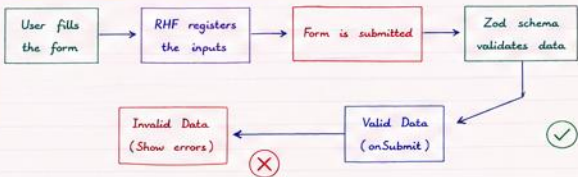
2. What is Zod ?

- Zod is a TypeScript-first schema validation library.
- Used to define schema and validate data.
- Works great with React Hook Form via [@hookform/resolvers/zod](https://github.com/hookform/resolvers/zod)

3. Installation

```
npm install react-hook-form @hookform/resolvers zod
```

4. How it Works ? (High Level Flow)



5. Basic Setup (Form with Zod)

Step 1: Import

```
import { useForm } from 'react-hook-form'
import { z } from 'zod'
import { zodResolver } from '@hookform/resolvers/zod'
```

Step 2: Create Zod Schema

```
const schema = z.object({
  name: z.string().min(2, 'Name must be at least 2 characters'),
  email: z.string().email('Invalid email address'),
  age: z.coerce.number().min(18, 'Must be 18 or above'),
  password: z.string().min(6, 'Password must be at least 6 chars'),
  confirm: z.string()
}).refine((data) => data.password === data.confirm, {
  message: 'Passwords do not match',
  path: ['confirm']
})
```

.min() =>
Minimum length
.email() =>
Valid email
.refine() =>
Custom validation

6. Use useForm() with zodResolver

```
const { register, handleSubmit, formState: { errors } } = useForm({
  resolver: zodResolver(schema)
})
```

resolver connects
Zod schema
with RHF

7. Register Inputs

```

<input type="text" {...register('name')} placeholder="Enter name" />
{errors.name && <p className="error">{errors.name.message}</p>}

<input type="email" {...register('email')} placeholder="Enter email" />
{errors.email && <p className="error">{errors.email.message}</p>}

<input type="number" {...register('age')} placeholder="Enter age" />
{errors.age && <p className="error">{errors.age.message}</p>}

<input type="password" {...register('password')} placeholder="Password" />
{errors.password && <p className="error">{errors.password.message}</p>}

<input type="password" {...register('confirm')} placeholder="Confirm Password" />
{errors.confirm && <p className="error">{errors.confirm.message}</p>}

```

errors
contains
validation
messages
from Zod

8. Handle Submit

```

const onSubmit = (data) => {
  console.log('Form Data.', data)
}

<form onSubmit={handleSubmit(onSubmit)} noValidate>
  {/* Inputs */}
  <button type="submit">Submit</button>
</form>

```

handleSubmit
will validate
using Zod
before calling
onSubmit

9. Complete Example (All in One)

```

import { useForm } from 'react-hook-form'
import { z } from 'zod'
import { zodResolver } from '@hookform/resolvers/zod'

const schema = z.object({
  name: z.string().min(2, 'Name must be at least 2 characters'),
  email: z.string().email('Invalid email address'),
  age: z.coerce.number().min(18, 'Must be 18 or above'),
  password: z.string().min(6, 'Password must be at least 6 chars'),
  confirm: z.string()
}).refine((data) => data.password === data.confirm, {
  message: 'Passwords do not match',
  path: ['confirm']
})

export default function MyForm() {
  const { register, handleSubmit, formState: { errors } } = useForm({
    resolver: zodResolver(schema)
  })
  const onSubmit = (data) => console.log(data)

  return (
    <form onSubmit={handleSubmit(onSubmit)} noValidate>
      {/* all inputs here using register('field') */}
      <button type="submit">Submit</button>
    </form>
  )
}

```

Zod validates
Automatically
before
submit

Common Input Validations (Zod)

Validation	Example	Meaning
z.string().min(3)	Minimum length	At least 3 characters
z.string().max(10)	Maximum length	At most 10 characters
z.string().email()	Email validation	Must be valid email
z.coerce.number().min(1)	Number + minimum	Must be >= 1
z.enum(['a', 'b'])	Enum validation	Value must be 'a' or 'b'
z.optional()	Optional field	Field is optional
.refine()	Custom validation	Custom rule

Build your
own rules
easily
with Zod



10. Useful RHF + Zod Tips

- ✓ Use `noValidate` on `<form>` to prevent native validation.
- ✓ Always show errors with `errors.field?.message`
- ✓ Use `defaultValues` in `useForm()` for controlled defaults.
- ✓ Use mode: 'onChange' for real-time validation.
- ✓ Zod runs only on submit (or trigger), not on every keypress.

Tip:
Better UX
comes from
good validation
strategy!



11. Default Values

```
const { register, handleSubmit, formState: { errors } } = useForm({
  resolver: zodResolver(schema),
  defaultValues: {
    name: '',
    email: '',
    age: '',
  }
})
```

`defaultValues`
pre-fills form
and prevents
uncontrolled
warnings

12. Modes (Validation Timing)

Mode	When it Validates
onSubmit (default)	When form is <u>submitted</u>
onChange	On every change
onBlur	When input loses focus
all	On change + blur + submit

Use
onChange
for live
validation

13. Trigger Manual Validation

```
const { trigger } = useForm()
// Validate specific field
trigger('email')
// Validate whole form
trigger()
```

Useful for
custom buttons
or conditional
validation

14. Quick Revision

- Install RHF + Zod + Resolver.
- Create Zod schema.
- Use `useForm({ resolver: zodResolver(schema) })`.
- Use `register()` for inputs.
- Show errors using `errors.field?.message`.
- Handle form with `handleSubmit()`.
- Enjoy clean, fast and type-safe forms!



Complete Flow



One Line Summary

React Hook Form + Zod = Fast Forms + Strong Validation + Great Developer Experience ❤️

★ Build Smart Forms, Ship Better Apps! 😊 ★

USE CALLBACK() & USE MEMO() - COMPLETE NOTES Page 1

1. What is useCallback() ?

- `useCallback()` returns a memoized (cached) version of a function.
- The function reference stays the same until the dependencies change.
- Prevents unnecessary re-creations of functions.

`useCallback`
= Cache
Function
Reference
☆

2. What is useMemo() ?

- `useMemo()` returns a memoized (cached) value.
- Re-computes the value only when dependencies change.
- Prevents expensive calculations on every render.

`useMemo`
= Cache
Computed
Value
☆

3. When to Use ?

Hook	Use When...
<code>useCallback()</code>	You want to cache/keep the same function reference between renders.
<code>useMemo()</code>	You want to cache expensive calculations and avoid re-computing.

4. Syntax

`useCallback()`

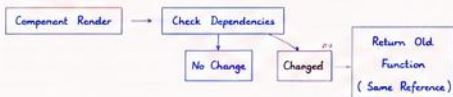
```
const memoizedFunction = useCallback(  
  () => {  
    // function logic  
  }, [dependencies]  
);
```

`useMemo()`

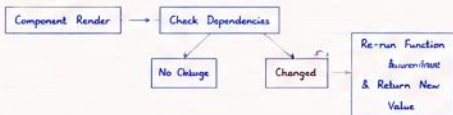
```
const memoizedValue = useMemo(  
  () => {  
    // expensive calculation  
    return result;  
  }, [dependencies]  
);
```

5. How it Works ?

`useCallback()`



`useMemo()`



Both hooks use dependency array to decide when to update or reuse.



6. Why Do We Need These Hooks ?

- ✓ Prevent unnecessary re-renders in child components.
- ✓ Improve performance.
- ✓ Avoid recreating functions / recalculating values on each render.
- ✓ Useful in large or complex applications.

Goal:
Optimize
Renders &
Improve
Performance
☆

7. Example - useCallback()

```
// Without useCallback()
function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = () => { // new function on every render
    console.log('Clicked', count);
  };

  return <Child handleClick={handleClick} />;
}

// With useCallback()
function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => { // same function until count changes
    console.log('Clicked', count);
  }, [count]);

  return <Child handleClick={handleClick} />;
}
```

Helps when
passing functions
to child
components
or using in
useEffect().



8. Example - useMemo()

```
// Without useMemo()
const total = items.reduce((acc, item) => acc + item.price, 0); // runs on every render

// With useMemo()
const total = useMemo(() => {
  return items.reduce((acc, item) => acc + item.price, 0);
}, [items]); // runs only when items changes
```

Great for
expensive
calculations



9. Dependency Array Rules

- [] → run once (only on mount)
- [value] → run when that value changes
- [value1, value2] → run when any of them changes
- No dependency array → runs on every render (Not recommended)

10. Dependant Points

- ✓ useCallback() returns a function reference.
- ✓ useMemo() returns a computed value.
- ✓ Don't use them everywhere. Use only when needed.
- ✓ Overuse can make code complex.

Use Smartly,
Not
Everywhere



11. Common Use Cases

<u>useCallback()</u>	<u>useMemo()</u>
• Passing functions to child components (with React.memo) • Functions used in useEffect() dependency array • Event handlers in optimized components	• Expensive calculations • Filtering, sorting large lists • Derived data from props/state • Prevent re-computation

Mini Summary

useCallback() → Memoize Function (Reference)

useMemo() → Memoize Value (Result)

Both help in performance optimization & prevent unnecessary renders.



12. useCallback() vs useMemo()

Feature	<u>useCallback()</u>	<u>useMemo()</u>
Returns	Memoized Function	Memoized Value
Purpose	Keep same function reference	Cache computed value
When to Use	When passing functions to child or <code>useEffect()</code>	When doing expensive calculations
Dependency	Array of dependencies	Array of dependencies
Re-render Prevention	Yes (Prevents child re-render)	Yes (Prevents re-calculation)
Example	<code>useCallback(() => {...}, [deps])</code>	<code>useMemo(() => {...}, [deps])</code>

13. Best Practices

- ✓ Use dependency array correctly.
- ✓ Avoid empty dependency [] unless needed.
- ✓ Don't put objects/arrays directly in dependency (`useMemo/useCallback` them).
- ✓ Keep functions and calculations outside component if possible.
- ✓ Profile performance before & after using.

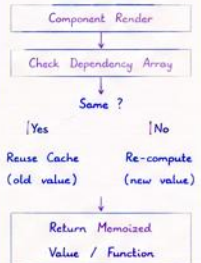
Correct usage
= Better
Performance
& Clean Code



14. Quick Revision

1. `useCallback()` → Cache function reference.
2. `useMemo()` → Cache computed value.
3. Both use dependency array.
4. Both help in performance optimization.
5. Use only when necessary.
6. Goal → Prevent unnecessary renders & calculations ☆

Complete Flow



15. Practical Example - Both Together

```

function Parent({ list }) {
  const [query, setQuery] = useState("");

  const filteredList = useMemo(() => {
    list.filter(item => item.name.toLowerCase().includes(query.toLowerCase()));
  }, [list, query]);

  const handleSearch = useCallback((e) => {
    setQuery(e.target.value);
  }, []); // stable function reference

  return (
    <div>
      <input value={query} onChange={handleSearch} placeholder="Search..." />
      <Child items={filteredList} />
    </div>
  );
}
  
```

Fitter only
when list
or query
changes.
Handler ref
stays same.
Performance
Improved



★ Remember → Optimize, Don't Over-Optimize ★

LAZY LOADING IN REACT - COMPLETE NOTES

1. What is Lazy Loading?

- Lazy Loading is a technique to load components or modules only when they are needed.
- It helps to reduce the initial bundle size and improve the performance of the app.
- In React, we use `React.lazy()` and `Suspense` for Lazy Loading.

Load only
what you need,
when you need.



2. Why Lazy Loading?

- ✓ Reduces Initial Bundle Size
- ✓ Faster Initial Load
- ✓ Better Performance
- ✓ Improves User Experience
- ✓ Loads Code on Demand

Less Code
in Initial Load
= Faster App



3. Syntax

`React.lazy()` : Used to dynamically import a component.

`Suspense` : Used to show a fallback (loading UI) while the component is loading.

Basic Syntax

```
const MyComponent = React.lazy(() =>
  import('./MyComponent')
);
```

With Suspense

```
<Suspense fallback={<div>Loading...</div>} >
  <MyComponent />
</Suspense>
```

4. How it Works?



5. Example - Simple Lazy Loading

```
import React, { Suspense, lazy } from 'react';
// Lazy loaded component
const About = lazy(() => import('./About'));
const Home = () => <h2>Home Page</h2>;
function App() {
  return (
    <div>
      <h1>My React App</h1>
      <Suspense fallback={<h3>Loading...</h3>} >
        <About />
      </Suspense>
    </div>
  );
}
export default App;
```

About component
will be loaded
only when needed.



6. Example - Lazy Loading with Routes

```
import React, { Suspense, lazy } from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';

const Home = lazy(() => import('./pages/Home'));
const About = lazy(() => import('./pages/About'));

function App() {
  return (
    <BrowserRouter>
      <Suspense fallback={<h3>Loading...</h3>} >
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}
export default App;
```

Components in
routes are
loaded only
when route
is visited.



7. Lazy Loading Named Exports

```
const MyComponent = lazy(() =>
  import('./MyComponent').then(module => ({
    default: module.MyComponent
  })))
);
```

Use when
component is a
named export.



8. Lazy Loading Multiple Components

```
const CompA = lazy(() => import('./CompA'));
const CompB = lazy(() => import('./CompB'));
const CompC = lazy(() => import('./CompC'));

// Use in Suspense as needed
```

Load multiple
components
on demand.



9. Combining Lazy Loading with React.memo()

```
const HeavyComponent = React.lazy(() => import('./HeavyComponent'));
const MemoisedHeavy = React.memo(HeavyComponent);

// Use inside Suspense
```

Good for
performance
optimisation



10. Best Practices

- ✓ Lazy load components that are not needed on initial render.
- ✓ Always wrap lazy-loaded components with `<Suspense>`.
- ✓ Use a meaningful fallback UI (spinner, skeleton, loader).
- ✓ Avoid lazy loading small or frequently used components.
- ✓ Combine with `React.memo()`, `useCallback()`, `useMemo()` for best results.

Lazy Load
Smartly,
Not
Everything



11. Common Use Cases

- Large Components (Dashboards, Reports)
- Route based Code Splitting.
- Modal / Popup Components
- Admin Panels / Settings Pages
- Third-party heavy libraries or components

Use Lazy
Loading where
it really
matters.



12. Lazy Loading vs Code Splitting

Feature	Lazy Loading	Code Splitting
Definition	Load components on demand	Split code into smaller bundles
Purpose	Delay loading of components	Reduce initial bundle size
How	React.lazy() + Suspense	Dynamic import() + Build tools (Webpack, Vite)
Result	Component loaded when needed	App is split into multiple bundles
Example	Lazy load About component	Split vendor, route, or feature code

13. Things to Remember

- Lazy Loading is a form of Code Splitting.
- It helps to improve performance and user experience.
- Always provide a good fallback UI.
- Don't overuse Lazy Loading.

Better
Performance
= Happy
Users



14. Complete Lazy Loading Flow



Load on
Demand,
Not in
Advance.



15. Quick Revision

- ✓ `React.lazy()` → Load component dynamically.
- ✓ `Suspense` → Show fallback while loading.
- ✓ Reduces bundle size
- ✓ Improves performance
- ✓ Use for routes, modals, heavy components
- ✓ Provide meaningful fallback UI



Complete Flow

Import Component
with `React.lazy()`

Wrap with
`<Suspense fallback=...>`

Fallback UI
Shown

Component
Loads

Component
Rendered

16. Practical Example - Lazy Loaded Modal

```

const Modal = lazy(() => import('./components/Modal'));

function App(){
  const {open, setOpen} = useState(false);
  return (
    <div>
      <button onClick={() => setOpen(true)}>Open Modal</button>
      {open & (
        <Suspense fallback={<p>Loading Modal...</p>}>
          <Modal onClose={() => setOpen(false)} />
        </Suspense>
      )}
    </div>
  );
}
  
```

Modal
component
is loaded
only when
opened.



LAZY LOADING = SMALLER BUNDLE SIZE + FASTER LOAD TIME
+ BETTER USER EXPERIENCE



LOAD SMART, PERFORM FAST, DELIVER BETTER! ☆

★ REDUX TOOLKIT (RTK) - TOPPER NOTES ★

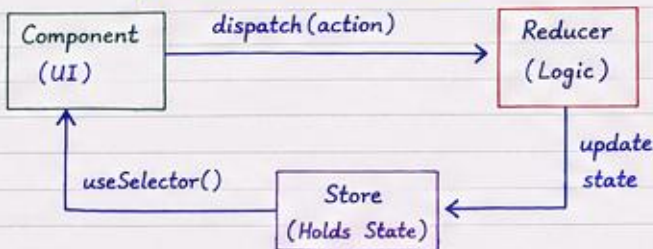
Page 1

1. What is Redux ?

Redux is a state management library for JavaScript apps.

It stores the entire application state in a single central store.

Redux Flow (Mental Model)



2. Core Building Block - SLICE

A slice is a collection of:

- initial state
- reducers
- generated action creators
- other reducer logic

★
`createSlice()`
from `@reduxjs/toolkit` is used
to create a slice.

3. Key Terms

- Store → central place that holds the state
- Slice → state + reducers + actions
- Reducer → function that decides how state changes based on action
- Action → object that describes what happened
- Dispatch → sends an action to the store
- Selector → reads data from the store
- Payload → data passed with an action
- Immer → RTK uses Immer internally to write "mutating" code safely.

4. createSlice() - Example (counterSlice.js)

```
import { createSlice } from "@reduxjs/toolkit"

const counterSlice = createSlice({
  name: 'counter',
  initialState: {
    value: 0,
    name: 'Kapil'
  },
  reducers: {
    increment: (state) => {
      state.value += 1
    },
    decrement: (state) => {
      if (state.value > 0) state.value -= 1
    },
    reset: (state) => {
      state.value = 0
    },
    changeName: (state, action) => {
      state.name = action.payload
    }
  }
})
```

We can write mutating code like `state.value++` because Immer handles immutability internally.

```
export const { increment, decrement, reset, changeName }
      = counterSlice.actions
export default counterSlice.reducer
```

5. Configure Store (store.js)

```
import { configureStore } from '@reduxjs/toolkit'
import counterReducer from '../features/counter/counterSlice'

const store = configureStore({
  reducer: {
    counter: counterReducer // key = "counter"
  }
})

export default store
```

State structure will be:

```
state.counter
  .value
state.counter
  .name
```

6. Provide Store to React App - Provider

```
import { Provider } from 'react-redux'
import store from './store'
import App from './App'
```

```
<Provider store={store}>
  <App />
</Provider>
```

Provider makes the store available to all components in the app.

7. Using Redux in Components

A. useSelector() - Read State

```
const value = useSelector(
  (state) => state.counter.value
)
```

useSelector always takes the latest state from the store.

B. useDispatch() - Send Action

```
const dispatch = useDispatch()
```

- dispatch(increment())
 - dispatch(decrement())
 - dispatch(reset())
 - dispatch(changeName("Rahul"))
- } Action Creators (from slice)

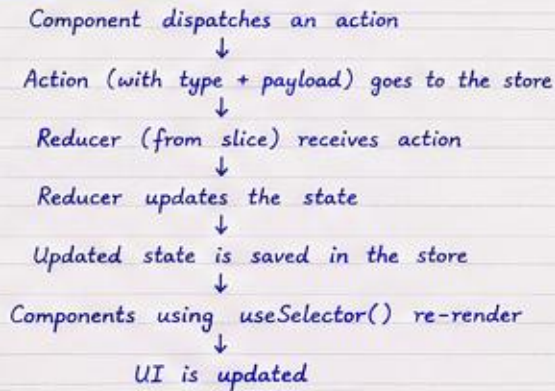
8. Action & Payload

When we pass some data while dispatching an action, it is available in reducer as action.payload

```
dispatch(changeName('Rahul')) → Reducer
                                state.name = action.payload
                                                ('Rahul')
```

★ Payload helps us send data with an action.

9. Complete Flow



★ Golden Rules

- ① READ → useSelector()
- ② WRITE → useDispatch()
- ③ ACTION → describes what happened
- ④ PAYLOAD → data with action
- ⑤ REDUCER → updates state
- ⑥ SLICE → state + reducers + actions
- ⑦ IMMER → safe mutating code in reducers

★ Understand the flow, not just the code. ★
That's the key to mastering Redux Toolkit! 😊

11. Quick Revision CheatSheet

- `createSlice()` → create slice (state + reducers + actions)
- `configureStore()` → create store and add reducers
- `Provider` → makes store available in app
- `useSelector()` → read state
- `useDispatch()` → send action
- `action.payload` → carries data
- `Immer` → write mutating code safely

Example Usage in Component

```
const value = useSelector(
  (state) => state.counter.value
)
const dispatch = useDispatch()

return (
  <div>
    <p>Count: {value}</p>
    <button onClick = {() => dispatch(increment())}></button>
    <button onClick = {() => dispatch(decrement())}></button>
    <button onClick = {() => dispatch(reset())}>Reset</button>
  </div>
)
```

Remember

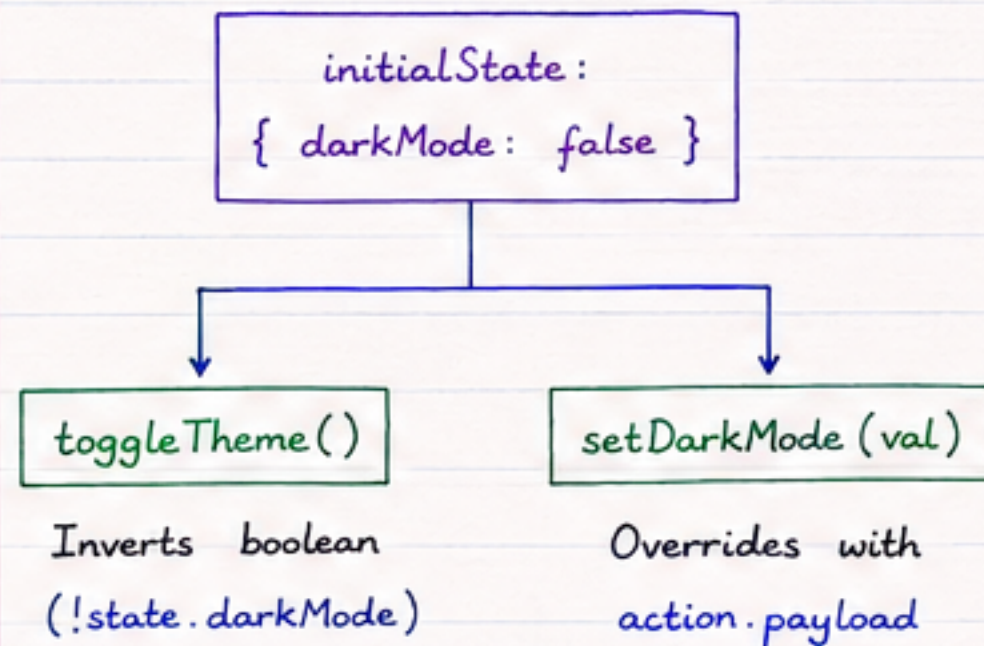
Redux Toolkit
reduces boilerplate
and makes Redux
simple, powerful
and enjoyable!



REDUX TOOLKIT (RTK) - THEME SLICE (themeSlice.js)

1. PURPOSE & OVERVIEW

- Role: Manages global UI theme state across the entire React app.
- Goal: Toggle between Dark Mode (true) and Light Mode (false), or explicitly assign a specific boolean state via payload.



MENTAL MODEL

Your UI doesn't store theme.
It asks Redux →
"What is the current theme?"
Redux answers →
"darkMode is true/false."
UI updates accordingly!



2. ANATOMY OF 'themeSlice.js'

createSlice() combines:

- ★ Slice Name: "theme"
(used as action prefix,
e.g., `theme/toggleTheme`)
- ★ Initial State: { darkMode: false }
- ★ Reducers: Case functions that update the state using Immer.js.

3. CODE IMPLEMENTATION

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  darkMode: false, // Default is Light Mode
};

const themeSlice = createSlice({
  name: "theme",
  initialState,
  reducers: {
    // Action 1: Toggle current boolean value
    toggleTheme: (state) => {
      state.darkMode = !state.darkMode;
    },
    // Action 2: Directly set to a specific boolean (true/false)
    setDarkMode: (state, action) => {
      state.darkMode = action.payload;
    },
  },
});

// Export Action Creators (for dispatching in UI)
export const { toggleTheme, setDarkMode } = themeSlice.actions;

// Export Reducer (for store registration)
export default themeSlice.reducer;
```

4. INTEGRATION CHECKLIST (Store → UI → DOM)

Step 1: Register in 'store.js'

```
reducer: {
  theme: themeReducer // State path: state.theme.darkMode
}
```

Step 2: Read & Dispatch in Component (ThemeToggle.jsx)

```
const darkMode = useSelector((state) => state.theme.darkMode);
const dispatch = useDispatch();
<button onClick={() => dispatch(toggleTheme())}> Toggle </button>
```

Step 3: Sync to HTML for Tailwind (App.jsx)

```
useEffect(() => {
  document.documentElement.classList.toggle("dark", darkMode);
}, [darkMode]);
```

5. COMMON PITFALLS & CHEAT SHEET

- ★ Redux Toolkit Uses Immer:
Direct mutations like `state.darkMode = ...` are completely safe inside `'createSlice'` reducers.
- ★ Action Type Strings:
 - `dispatch(toggleTheme())`
→ { type: `'theme/toggleTheme'` }
 - `dispatch(setDarkMode(true))`
→ { type: `'theme/setDarkMode'`, payload: true }
- ★ Tailwind Class Strategy:
Don't forget `'darkMode: 'class' (or 'selector')` in your CSS configuration for the `'dark:'` utility variants to trigger.

QUICK RECAP

ACTIONS

```
toggleTheme()
  → Flip (true ↔ false)

setDarkMode(val)
  → Set to specific value (true/false)
```

STATE PATH

`state.theme.darkMode`

Single source of truth for theme in the app!



REDUX TOOLKIT (RTK) ASYNC - CHEAT SHEET ★

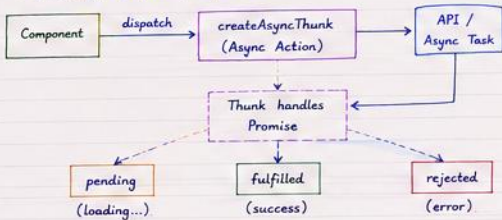
Page 1

1. Why Async in Redux?

- APIs, DB calls, timeouts → all are asynchronous.
- We need to handle → loading, success, error properly in the store.

2. The Hero - createAsyncThunk()

- Built-in function in RTK to handle async logic.
- Automatically generates 3 actions →
— pending, fulfilled, rejected.



3. createAsyncThunk() - Syntax

```
const asyncThunk = createAsyncThunk(  
  "sliceName/thunkName", // Action type prefix  
  async (arg, thunkAPI) => { // Async logic  
    return data;  
  }  
);
```

arg → any value
you pass while
dispatching.
thunkAPI →
has helpers
like rejectWithValue

4. Example - Fetch Users from API

```
// usersThunk.js
export const fetchUsers = createAsyncThunk(
  'users/fetchUsers',
  async (... , thunkAPI) => {
    try {
      const res = await fetch('https://jsonplaceholder.typicode.com/users');
      if (!res.ok) throw new Error('Failed to fetch');
      const data = await res.json();
      return data;
    } catch (err) {
      return thunkAPI.rejectWithValue(err.message);
    }
  }
);
```

We return
data on success.
On error →
rejectWithValue()
with message.

5. Handle in Slice - extraReducers

```
const usersSlice = createSlice({
  name: 'users',
  initialState: {
    users: [],
    loading: false,
    error: null
  },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.loading = false;
        state.users = action.payload;
      })
      .addCase(fetchUsers.rejected, (state, action) => {
        state.loading = false;
        state.error = action.payload;
      });
  }
});
```

3 Auto Actions

1. users/fetchUsers/pending
2. users/fetchUsers/fulfilled
3. users/fetchUsers/rejected

State Shape for Async

loading → boolean

users → []

error → string | null

6. Use in Component

```
// ExampleComponent.jsx
const MyComponent = () => {
  const dispatch = useDispatch();
  const { users, loading, error } = useSelector(
    (state) => state.users
  );

  useEffect(() => {
    dispatch(fetchUsers());
  }, [dispatch]);

  if (loading) return <p>Loading....</p>;
  if (error) return <p style={{color: 'red'}}>{error}</p>;

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
};
```

dispatch
the thunk
like a normal
action!

7. Dispatching with Parameters

```
// Pass parameter while dispatching
dispatch(fetchUserById(5)); // 5 → will be available in arg
dispatch(fetchUsers({ page: 1 })); // object also ok
```

8. Error Handling Best Practice

- Always use `rejectWithValue()` to send meaningful error.
- It will be available in `action.payload` in rejected case.

```
return thunkAPI.rejectWithValue("Something went wrong!");
```

9. rejectWithValue() vs throw new Error()

<u>throw new Error()</u>	<u>rejectWithValue()</u>
Sends error in <code>action.error</code> (less control)	Sends custom message in <code>action.payload</code> .
<code>action.error.message</code> will have the message.	You have full control over what error is stored.
Not recommended for expected errors (API errors).	Best practice for API error handling.

10. Other Useful ThunkAPI Helpers

- `thunkAPI.dispatch` → dispatch other actions
- `thunkAPI.getState()` → access current state
- `thunkAPI.rejectWithValue(value)` → handle errors
- `thunkAPI.fulfillWithValue(value, meta?)` → return custom fulfilled value

★ Async Flow Recap ★

`dispatch(thunk)` → pending → API Call → fulfilled (success)
 ↓
 rejected (error)

11. Complete Example in One Place

```
// usersSlice.js (Full Example)
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";

export const fetchUsers = createAsyncThunk(
  'users/fetchUsers',
  async (_, thunkAPI) => {
    try {
      const res = await fetch('https://jsonplaceholder.typicode.com/users');
      if (!res.ok) throw new Error('Failed to fetch');
      return await res.json();
    } catch (err) {
      return thunkAPI.rejectWithValue(err.message);
    }
  }
);

const usersSlice = createSlice({
  name: 'users',
  initialState: { users: [], loading: false, error: null },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => {
        state.loading = true; state.error = null;
      })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.loading = false; state.users = action.payload;
      })
      .addCase(fetchUsers.rejected, (state, action) => {
        state.loading = false; state.error = action.payload;
      });
  }
});

export default usersSlice.reducer;
```

Key Points

- ★ createAsyncThunk handles the async logic.
- ★ It creates 3 actions automatically.
- ★ Handle them in extraReducers.
- ★ Always manage loading & error in state.

Practice more
with different
APIs!



★ QUICK REVISION ★

1. Use `createAsyncThunk` for async logic.
2. It creates 3 actions → pending, fulfilled, rejected.
3. Handle in `extraReducers` using `builder`.
4. Manage loading, success data & error in state.
5. Use `rejectWithValue()` for meaningful errors.
6. Dispatch thunk like a normal action.
7. Access data in component using `useSelector()`.
8. Keep state shape consistent.

STATE SHAPE TEMPLATE

```
{  
  data: [],  
  loading: false,  
  error: null  
}
```

Remember
this
always!



1. What is RTK Query?

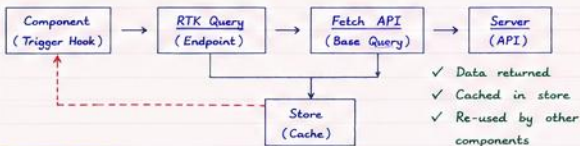
- RTK Query is the data fetching and caching tool built inside Redux Toolkit.
- It simplifies API calls, caching, loading states, error handling, polling, pagination etc.
- Eliminates the need for extra libraries like axios + react-query in most cases.
- Built on top of Redux Toolkit + Immer.

Less code
Auto caching
Loading & error handling built-in
Great DX 🍌

2. Core Concepts

- **Endpoint** → A description of how to fetch (or mutate) data.
- **Query** → For fetching data (GET).
- **Mutation** → For changing data (POST, PUT, DELETE).
- **Cache** → Stores the result of queries in Redux store.
- **Tag** → Used for cache invalidation & refetching.
- **Re-usable Hooks** → Auto generated hooks for each endpoint.

3. How RTK Query Works (High Level Flow)



4. Basic Setup

Step 1: Install (if needed)

```
npm i @reduxjs/toolkit react-redux
```

Step 2: Create an API Slice (api/postsApi.js)

```
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react'

export const postsApi = createApi({
  reducerPath: 'postsApi',
  baseQuery: fetchBaseQuery({ baseUrl: 'https://jsonplaceholder.typicode.com' }),
  endpoints: (builder) => ({
    getPosts: builder.query({
      query: () => '/posts',
    }),
  }),
})
```

createApi creates a slice with reducer + middleware automatically

Step 3: Add to Store (store.js)

```
import { configureStore } from '@reduxjs/toolkit'
import { postsApi } from './api/postsApi'

export const store = configureStore({
  reducer: {
    [postsApi.reducerPath]: postsApi.reducer,
  },
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(postsApi.middleware),
})
```

Don't forget to add middleware!

Step 4: Provide Store

```
<Provider store={store}>
  <App />
</Provider>
```

Now RTK Query is ready to use in the app 🍌

5. Endpoints - Query vs Mutation

Query (GET)	Mutation (POST/PUT/DELETE)
- Read / fetch data - Cached automatically - Auto refetch on certain events - Uses <code>builder.query()</code>	- Change / update data - Does not cache result - You usually invalidate tags - Uses <code>builder.mutation()</code>

6. Multiple Endpoints Example

```
export const postsApi = createApi({
  reducerPath: 'postsApi',
  baseQuery: fetchBaseQuery({ baseUrl: 'https://jsonplaceholder.typicode.com' }),
  tagTypes: ['Post'],
  endpoints: (builder) => ({
    getPosts: builder.query({
      query: () => '/posts',
      providesTags: (result) =>
        result
          ? [
              ...result.map(({ id }) => ({ type: 'Post', id })),
              { type: 'Post', id: 'LIST' },
            ]
          : [{ type: 'Post', id: 'LIST' }],
    }),
    addPost: builder.mutation({
      query: (newPost) => ({
        url: '/posts',
        method: 'POST',
        body: newPost,
      }),
      invalidatesTags: [{ type: 'Post', id: 'LIST' }],
    }),
  }),
})
```

providesTags → Tells what data this query provides

invalidatesTags → Marks cache as stale & triggers refetch.

7. Auto Generated Hooks

Endpoint Type	Example Endpoint	Auto Generated Hook
Query	<code>getPosts</code>	<code>useGetPostsQuery()</code>
Mutation	<code>addPost</code>	<code>useAddPostMutation()</code>

- Hooks are generated based on endpoint name.
- Use them directly in components.

8. Using in Component

Example - Fetching Data

```
import { useGetPostsQuery } from './api/postsApi'

function Posts() {
  const { data, isLoading, isError, error } = useGetPostsQuery()
  if (isLoading) return <p>Loading...</p>
  if (isError) return <p>Error: {error.toString()}</p>
  return (
    <ul>
      {data?.map(post) => (
        <li key={post.id}>{post.title}</li>
      )}
    </ul>
  )
}
```

data → result

isLoading → loading state

isError → error state

error → error object

9. Using Mutation in Component

```
import { useAddPostMutation } from './api/postsApi'

function AddPost() {
  const [addPost, { isLoading, isSuccess, isError }] = useAddPostMutation()

  const handleAdd = () => {
    addPost({
      title: 'foo',
      body: 'bar',
      userId: 1,
    })
  }

  return (
    <div>
      <button onClick={handleAdd} disabled={isLoading}
        {isLoading ? 'Adding...' : 'Add Post'}
      </button>
      {isSuccess && <p>✅ Post Added!</p>}
      {isError && <p>❌ Failed to add post</p>}
    </div>
  )
}
```

Mutation returns a trigger function + status flags.

10. Cache & Invalidation

- Queries are cached based on endpoint + parameters.
- By default cache is kept as long as component is mounted.
- Use `invalidatesTags` in mutation to refetch related queries.
- You can also use `refetch()` manually.

11. Useful Options in Endpoints

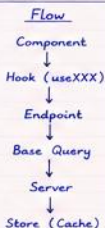
Option	Purpose
<code>query</code>	Defines the API endpoint.
<code>providesTags</code>	Defines what data this query provides.
<code>invalidatesTags</code>	Invalidates related cache.
<code>transformResponse</code>	Modifies the response data.
<code>keepUnusedDataFor</code>	Time (in sec) to keep data in cache after component unmount.

12. Benefits

- ✓ Less boilerplate code
- ✓ Built-in caching
- ✓ Auto refetching
- ✓ Loading & error handling
- ✓ Pagination, polling, refetch on focus
- ✓ Tag based cache invalidation
- ✓ Great Developer Experience.

13. Quick Revision

1. `createApi()` → Create API slice.
2. Define endpoints with `builder.query / builder.mutation`.
3. Add reducer + middleware to store.
4. Use auto generated hooks in components.
5. Enjoy caching, refetching & less boilerplate! 🎉



1. What is Zustand ?

- Zustand is a small, fast and scalable state management library for React.
- It uses a simple store based approach.
- No Providers, No Complex Setup.
- Combines simplicity of useState with power of Redux.

Small
Fast
Simple



2. Core Concepts

- **Store** → Central place to hold state.
- **useStore()** → Hook to access store.
- **get()** → Read state outside components.
- **set()** → Update state.
- **Action** → Functions that update state.



3. Basic Store Example

```
import { create } from 'zustand'
// create a store
const useCounterStore = create((set) => ({
  count: 0,
  increment: () => set((state) => ({ count: state.count + 1 })),
  decrement: () => set((state) => ({ count: state.count - 1 })),
  reset: () => set({ count: 0 })
}))
```

Note:

- **set()** merges state by default.
- You can also return an object to update.

Explanation

- **count** → initial state
- **increment()** → increases count
- **decrement()** → decreases count
- **reset()** → sets count to 0

Looks like useState,
Works like Redux !

4. Using Store in Components

```
import React from 'react'
import useCounterStore from './store/counterStore'

function Counter() {
  const count = useCounterStore((state) => state.count)
  const { increment, decrement, reset } = useCounterStore()
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={increment}>+1</button>
      <button onClick={decrement}>-1</button>
      <button onClick={reset}>Reset</button>
    </div>
  )
}
```

useStore()
subscribes only
to the used
state (Optimized
re-renders).

5. Advanced: Using get()

// get() helps to read current state inside actions

```
const useUserStore = create((set, get) => ({
  name: 'Kapil',
  updateName: (newName) => set({ name: newName }),
  capitalizeName: () => {
    const current = get().name
    set({ name: current.toUpperCase() })
  }
}))
```

get()
reads current
state inside
actions.

6. Persist Middleware (State Persistence)

```
import { create } from 'zustand'
import { persist } from 'zustand/middleware'

const usePersistStore = create(
  persist(
    (set) => ({
      count: 0,
      increment: () => set((state) => ({ count: state.count + 1 })),
    }),
    { name: 'my-store' } // store name in localStorage
  )
)
```

State will
be saved in
localStorage
automatically.

7. Devtools Middleware (Redux DevTools Support)

```
import { devtools } from 'zustand/middleware'

const useDevtoolsStore = create(
  devtools((set) => ({
    count: 0,
    increment: () => set((state) => ({ count: state.count + 1 })),
  })),
)
```

Check state
changes in
Redux
DevTools.

8. Combining Middlewares

```
import { create } from 'zustand'
import { persist } from 'zustand/middleware'
import { devtools } from 'zustand/middleware'

const useStore = create(
  devtools(
    persist(
      (set) => ({
        count: 0,
        increment: () => set((state) => ({ count: state.count + 1 })),
      }),
      { name: 'my-store' }
    )
  )
)
```

Order
devtools(
 persist(
 yourStore
)
)

Outer → devtools
Inner → persist

9. Immutable Updates

- Zustand allows mutable style updates.
- But always return new object (best practice).

Always
return new
object / array.

```
const useTodoStore = create((set) => ({
  todos: [],
  addTodo: (text) => set((state) => ({
    todos: [...state.todos, { id: Date.now(), text: text, done: false }]
  })),
  toggleTodo: (id) => set((state) => ({
    todos: state.todos.map(t) =>
      t.id === id ? {...t, done: !t.done} : t
  })),
}))
```

10. Subscribing Outside Components

```
// Useful for non-React logic
const unsub = useCounterStore.subscribe((state) =>
  console.log('Count changed.', state.count)
)

// later call to stop
unsub()
```

subscribe()
to listen
state changes
outside UI.

11. Advantages of Zustand

- ✓ Very small size (~1KB)
- ✓ Simple API
- ✓ No Providers
- ✓ Fast performance
- ✓ Works well with TypeScript
- ✓ DevTools support
- ✓ Middlewares for advanced needs

12. When to Use Zustand ?

- Small to medium applications.
- When you want simple and lightweight state management.
- When Redux feels heavy.
- When you want good performance + less boilerplate.

QUICK REVISION

Read State

```
const value = useStore(
  (state) => state.value
)
```

Update State

```
set((state) => ({
  value: newValue
}))
```

Outside Component

```
const value = useStore.getState()
```

Subscribe

```
const unsub = useStore.subscribe(
  (state) => console.log(state)
)

unsub() // stop
```

Flow

Action



set()



Store Update



Subscriber
(Component)
re-renders

★ Zustand = Simplicity + Performance + Power ★

REDUX TOOLKIT (RTK) vs ZUSTAND

Page 1

BASICS COMPARISON

★ 1. What are they?

- RTK → Official, opinionated, batteries-included state management for large-scale apps.
- Zustand → Small, simple, unopinionated store for React. Uses hook-based API.

Both
Manage
State!



★ 2. Core Philosophy

<u>Redux Toolkit (RTK)</u>	<u>Zustand</u>
Follow structure and patterns Best for predictability and maintainability.	Minimal API Do what you need, no extra rules.

★ 3. Boilerplate Code

RTK → More setup

- Store
- Slice
- Actions
- Provider

Zustand → Very less setup

- Create store
- Use store in component

★ 4. Setup Example

RTK Setup Flow

```
createSlice()
  ↓
configureStore()
  ↓
<Provider store={store}>
  ↓
useSelector / useDispatch()
```

Zustand Setup Flow

```
create()
  ↓
useStore()
  ↓
Done ✓
```

★ 5. State Access in Components

RTK

```
const value = useSelector(
  (state) => state.slice.value
)
```

```
const dispatch = useDispatch()
```

Zustand

```
const value = useStore(
  (state) => state.value
)
```

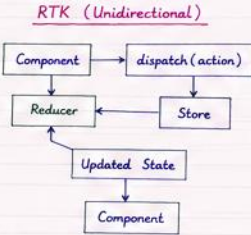
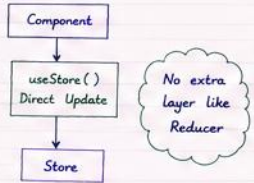
Direct hook usage!
No dispatch needed
for simple updates.



★ 6. State Update

<u>RTK</u>	<u>Zustand</u>
<code>dispatch(increment())</code> <code>dispatch(setValue(10))</code>	<code>const increment =</code> <code>useStore((state) → state.increment)</code> <code>increment()</code>
Updates are done by dispatching actions.	Call function directly from the store.

★ 7. Data Flow

Zustand

★ 8. Best For

<u>RTK</u>	<u>Zustand</u>
Large applications	Small to medium applications
Complex state logic	Simple state logic
Teams who like structure	Teams who like simplicity and flexibility
Need devtools, middleware, RTK Query, etc.	Need lightweight state management

★ 9. Middleware & DevTools

<u>RTK</u>	<u>Zustand</u>
✓ Built-in support	✓ Devtools middleware
✓ Rtk Toolkit DevTools	✓ Persist middleware
✓ Middleware ecosystem	✓ Very flexible

★ 10. Learning Curve

<u>RTK</u>
Medium

Zustand is easier for beginners.

<u>Zustand</u>
Easy

★ 11. Performance

Both are fast ⚡ but Zustand is slightly lighter (smaller bundle size).



★ 12. Feature Comparison

<u>Feature</u>	<u>RTK</u>	<u>Zustand</u>
State Management	Yes	Yes
Boilerplate	More	Very Less
Setup	Complex	Simple
DevTools	Built-in	Middleware
Middleware	Built-in support	Manual add
Async Logic (RTK Query)	Yes	No (Manual)
Persist State	Yes	Yes (Middleware)
Best For	Large & Complex Apps	Small & Medium Apps
Learning Curve	Medium	Easy
Flexibility	Structured	Highly Flexible

★ 13. When to use What?

Use RTK when:

- Building large scale app.
- Need strict structure.
- Need RTK Query for API.
- Working in a big team.

VS

Use Zustand when:

- Building small / medium app.
- Need simple global state.
- Want less boilerplate.
- Like flexible approach.

★ 14. Quick Notes

- ✓ RTK → Structure + Convention + Powerful.
Good for long-term large projects.
- ✓ Zustand → Simplicity + Flexibility + Lightweight.
Good for quick and modern apps.

☑ Both are great!
Choose based on project size and team preference.



★ 15. One Line Summary

RTK

More boilerplate,
More power,
Better for big apps.

VS

Zustand

Less code,
More flexibility,
Better for simple apps.

★ RIGHT TOOL, RIGHT TIME, RIGHT PROJECT ★